

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital

Ingeniería de Software

Aplicaciones Web

*del navegador al servidor,
del prototipo al despliegue*

Un compendio construido alrededor de un proyecto real:
BiblioUTEQ — Sistema de Gestión de Biblioteca Universitaria

Autor: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Asignatura: Aplicaciones Web (5.º nivel, 190 h, 5 créditos)

Período: PPA 2026–2027

Versión: 4.0 (compendio modular, 4 unidades)

Aplicaciones Web — Compendio para Ingeniería de Software (UTEQ).

Copyright © 2026 Gleiston Cicerón Guerrero Ulloa. Todos los derechos reservados para fines editoriales; distribución interna con fines docentes en la Universidad Técnica Estatal de Quevedo.

Las marcas comerciales mencionadas (Java, Spring, Angular, PostgreSQL, entre otras) pertenecen a sus respectivos titulares y se citan con propósito académico.

Compuesto en L^AT_EX con la clase `book`, listados con `listings` y cajas temáticas con `tcolorbox`.
Bibliografía gestionada con BibL^AT_EX.

Prólogo

«Enseñar a construir es enseñar a decidir;
la web es el terreno donde las decisiones
son visibles al instante.»

— *Nota del autor*

Este libro nació de una necesidad concreta del aula. Durante varios períodos, el aprendizaje de Ingeniería de Software transitaba por HTML5, CSS3 y JavaScript en cuatro semanas, saltaba a un lenguaje del servidor sin puente conceptual, y llegaba al proyecto final sin un hilo que conectara todo lo estudiado. El material estaba ahí, disperso en guías sueltas; faltaba un tejido que lo uniera.

El compendio que el lector tiene entre manos ofrece ese tejido en dos direcciones. Primero, un **proyecto conductor** —BiblioUTEQ, un sistema de gestión de biblioteca universitaria— atraviesa las cuatro unidades: la Unidad I diseña su interfaz, la Unidad II construye su servidor, la Unidad III lo pone a hablar con la base de datos siguiendo patrones, y la Unidad IV lo integra en una arquitectura completa que se despliega en Docker y se defiende como Proyecto de Fin de Curso (PFC). Cada capítulo agrega piezas al mismo repositorio; nada queda huérfano.

Segundo, cada tema se presenta con el mismo método práctico: primero se plantea el problema concreto que se va a resolver —por qué le duele al desarrollador, qué síntoma manifiesta—; después se explica la técnica que lo resuelve, con la referencia bibliográfica que la respalda; después se muestra un ejemplo pequeño pero completo con su código, y al final se explica cómo esa pieza encaja en BiblioUTEQ. La caja de *Problema* abre; la caja de *Definición* explica; el *Listado* muestra; el texto siguiente cierra explicando cómo se resolvió.

Al cierre de cada unidad, una **guía de entrega del PFC** indica exactamente qué piezas debe subir el equipo al repositorio de GitHub, con qué etiqueta Git, en qué formato, y bajo qué rúbrica será evaluado. Las entregas están diseñadas para acumular evidencia con la robustez requerida por revistas científicas de alto impacto, sin distraer con esa meta al equipo que las produce; para el aprendiz, el objetivo declarado es entregar un sistema que funciona, y esa es la única meta visible.

Gleiston Cicerón Guerrero Ulloa, Ph.D.
Quevedo, julio de 2026

Índice general

Prólogo	i
Encuadre	1
Encuadre	3
1. El proyecto BiblioUTEQ	3
Resultado de aprendizaje del encuadre	3
1.1. Contexto de la asignatura y del compendio	3
1.1.1. La asignatura Aplicaciones Web	3
1.1.2. El aprendiz al que se dirige el compendio	4
1.1.3. Cómo se lee este libro	4
1.2. El dominio: gestión de biblioteca universitaria	4
1.2.1. Actores del sistema	4
1.2.2. Reglas de negocio principales	5
1.2.3. Restricciones no funcionales	6
1.3. Modelo conceptual de BiblioUTEQ	6
1.3.1. Entidades y sus responsabilidades	6
1.3.2. Casos de uso críticos	7
1.4. Arquitectura general y decisiones iniciales	7
1.4.1. Estilo arquitectónico: cliente monopágina más API REST	8
1.4.2. Pilas del servidor aceptadas por la asignatura	8
1.4.3. Base de datos: PostgreSQL como estándar	8
1.5. Estructura del repositorio (monorepo)	9
1.5.1. Convención de carpetas	9
1.5.2. Convención de nombres de paquetes	10
1.5.3. Convención de ramas y etiquetas Git	10
1.6. Herramientas del entorno de desarrollo	10
1.6.1. IDE de referencia y editores auxiliares	11
1.6.2. Servicios locales	11
1.6.3. Comandos de arranque	11
1.7. Cómo se organiza cada capítulo del libro	11
1.7.1. El método del libro	11
1.7.2. Cajas temáticas y su significado	12
1.8. Cierre del encuadre	12

I	Fundamentos del cliente web	13
	Unidad I — En preparación	15
II	Unidad 2 — En preparación	17
	Unidad 2 — En preparación	19
III	Unidad 3 — En preparación	21
	Unidad 3 — En preparación	23
IV	Unidad 4 — En preparación	25
	Unidad 4 — En preparación	27
	Bibliografía	29
	2. Bibliografía	29

Índice de figuras

1.1. Diagrama entidad-relación inicial de BiblioUTEQ. Las cardinalidades finales se refinan al pasar del modelo conceptual al físico en la Unidad III.	6
1.2. Estructura de carpetas de nivel superior del monorepo BiblioUTEQ.	9

Índice de tablas

1.1. Reglas de negocio principales de BiblioUTEQ.	5
1.2. Herramientas del entorno de desarrollo con versiones de referencia.	10

Lista de listados

Encuadre

El proyecto BiblioUTEQ

«Un ingeniero no aprende a construir sistemas leyendo listas de tecnologías, sino resolviendo un problema real de principio a fin.»

— *Principio metodológico del compendio*

Resultado de aprendizaje del encuadre

Al concluir la lectura de este capítulo, el aprendiz reconoce el dominio de negocio de BiblioUTEQ, identifica las entidades y reglas que gobiernan el sistema, entiende la estructura del monorepo que soportará el trabajo de las cuatro unidades, y sabe qué rol asume en cada uno de los cuatro cortes del PFC. En otras palabras, deja de leer un libro y comienza a habitar un proyecto.

1.1 Contexto de la asignatura y del compendio

Aplicaciones Web es una asignatura de quinto nivel de la carrera de Ingeniería de Software de la Universidad Técnica Estatal de Quevedo, con carga de cinco créditos distribuidos en 190 horas totales a lo largo de dieciocho semanas del período académico PPA 2026–2027. Su ubicación en la malla no es casual: el estudiante llega con dominio de Java, algoritmos, bases de datos relacionales y modelado orientado a objetos, y sale —si el curso cumple su propósito— capaz de diseñar, construir, asegurar y desplegar un sistema completo del lado servidor con su cliente enriquecido.

El compendio está organizado en cuatro unidades que reproducen el arco pedagógico natural de la disciplina: los fundamentos del cliente (Unidad I), las tecnologías del servidor (Unidad II), los datos y sus patrones (Unidad III), y la arquitectura completa con servicios web y despliegue (Unidad IV). Cada unidad cierra con una entrega del Proyecto de Fin de Curso —el PFC— que acumula, por sedimentación, la totalidad del sistema aprendido.

1.1.1 La asignatura Aplicaciones Web

La asignatura persigue un objetivo profesional que trasciende lo procedimental: formar un desarrollador que no solo escriba código funcional, sino que decida qué patrón arquitectónico aplica a cada problema, qué trade-off asume al elegir un stack por sobre otro, y qué evidencias documentar para que la decisión quede trazable. Esta segunda capa —el porqué detrás del cómo— distingue a un profesional de un usuario avanzado del framework de moda.

1.1.2 El aprendiz al que se dirige el compendio

El lector supuesto es un estudiante que ha aprobado Programación Orientada a Objetos y Bases de Datos en niveles anteriores, y que se encuentra por primera vez con la construcción de aplicaciones distribuidas. No se asume experiencia previa con HTML5, CSS3 o JavaScript más allá de lo que resulta de una curiosidad técnica normal; sí se asume dominio del ecosistema Java básico —clases, colecciones, excepciones, JDBC— y familiaridad con SQL estándar.

El aprendiz que este libro imagina es alguien que quiere ver funcionar el sistema, entiende que el fallo es parte del proceso, y no espera recetas mágicas que compensen la ausencia de comprensión. Con esa disposición, el compendio funciona; sin ella, ningún libro lo hace.

1.1.3 Cómo se lee este libro

Cada capítulo sigue una estructura repetitiva de propósito. Se abre con el problema concreto que se va a resolver, planteado en una caja de *Problema*; se continúa con una explicación técnica sostenida en referencias verificables; se muestra un listado de código con la ruta del archivo dentro del proyecto y una explicación posterior de cómo el listado resuelve el problema planteado; y se cierra con una nota que conecta la pieza recién construida con el estado global del proyecto BiblioUTEQ.

El lector que persiga solo el código tenderá a saltar de listado en listado. Esa lectura, aunque tentadora, deja fuera el porqué y hace pesado el mantenimiento posterior del sistema. La recomendación del autor es leer los textos que enmarcan cada listado: son cortos por decisión de diseño, y llevan las citas donde el aprendiz puede ampliar cada tema en su fuente original.

1.2 El dominio: gestión de biblioteca universitaria

Problema a resolver

La biblioteca central de una universidad pública ecuatoriana atiende alrededor de tres mil usuarios diarios en períodos académicos activos, con un catálogo que supera los cuarenta mil títulos, catorce mil de los cuales tienen más de un ejemplar. El sistema anterior es una hoja de cálculo compartida y un cuaderno de préstamos físico. La dirección de biblioteca necesita un sistema web que registre préstamos, controle vencimientos, prevenga pérdidas y produzca reportes verificables para la auditoría institucional.

La gestión de una biblioteca universitaria concentra, en un dominio conocido por cualquier estudiante, todas las tensiones que aparecen en sistemas de información más complejos: identidad de personas con roles diferenciados, inventario con estados que transitan, reglas de negocio con excepciones bien definidas, reportes que agregan datos históricos, y una superficie de auditoría sobre operaciones sensibles. Precisamente por su cercanía cotidiana —todo estudiante sabe qué es pedir un libro prestado— resulta pedagógicamente idóneo para trabajar los patrones clásicos de una aplicación empresarial [1, 2].

1.2.1 Actores del sistema

El sistema reconoce cinco actores primarios cuyos derechos y responsabilidades son distintos entre sí.

El **lector** —estudiante, docente o administrativo— se autentica en la aplicación, consulta el catálogo, reserva títulos, revisa sus préstamos activos y recibe notificaciones de vencimiento. No modifica el catálogo ni gestiona a otros usuarios; su alcance es su propia cuenta.

El **bibliotecario** atiende el mostrador. Registra préstamos físicos y devoluciones, marca ejemplares como perdidos o dañados, resuelve conflictos entre reservas y disponibilidad, y consulta el histórico de un lector específico cuando así lo requiere la operación diaria.

El **catalogador** da de alta títulos y ejemplares. Es un rol especializado que trabaja más con la calidad del catálogo que con el mostrador; corrige metadatos, fusiona duplicados y gestiona la clasificación Dewey o el sistema temático institucional [3].

El **director de biblioteca** accede a reportes agregados, revisa indicadores de rotación, morosidad y cobertura temática, y aprueba baja de ejemplares o depuración de fichas históricas. No opera el mostrador ni carga catalogación masiva.

El **administrador del sistema** gestiona usuarios y roles, ejecuta migraciones de esquema y supervisa la salud técnica de la aplicación. Se documenta como actor de sistema para que las decisiones de seguridad —por quién pasa la creación de un rol, por ejemplo— queden explícitas en los diagramas de casos de uso.

1.2.2 Reglas de negocio principales

La biblioteca opera sobre un conjunto de reglas que la aplicación debe hacer cumplir de forma automática y auditable. La Tabla 1.1 resume las más importantes; cada una da lugar, más adelante en el libro, a un caso de uso o a una regla implementada en el servidor.

Tabla 1.1: *Reglas de negocio principales de BiblioUTEQ.*

Regla	Enunciado
R1 — Límite de préstamos activos	Un lector tiene simultáneamente hasta 3 préstamos activos si es estudiante, hasta 5 si es docente y hasta 4 si es administrativo.
R2 — Plazo de préstamo	El plazo es de 7 días calendario para estudiantes, 14 días para docentes y 10 días para administrativos, contados desde la fecha de retiro.
R3 — Renovación	Un préstamo se renueva una sola vez si el ejemplar no tiene reservas y el lector no tiene multas pendientes.
R4 — Multa por retraso	Cada día de retraso genera una multa de USD 0,50 hasta un tope de USD 20,00 por ejemplar. La multa se aplica al momento de la devolución.
R5 — Reserva y cola	Un lector reserva un título cuando todos sus ejemplares están prestados; la cola es FIFO y expira si el lector no retira el ejemplar dentro de 48 horas de su disponibilidad.
R6 — Bloqueo por multa	Un lector con al menos una multa impaga no puede realizar préstamos nuevos ni reservas.
R7 — Auditoría	Toda operación que altere el estado de un ejemplar o de una multa se registra en una bitácora que conserva usuario, fecha, valor previo y valor posterior.

Estas siete reglas conforman el núcleo del comportamiento del sistema, y el libro las trata explícitamente como **invariantes del dominio**: propiedades que el software debe garantizar independientemente del camino de ejecución que las provoque [4]. La característica de invariante —frente a la de simple validación— justifica más adelante decisiones específicas sobre dónde y cómo se comprueban en el código.

1.2.3 Restricciones no funcionales

Además de las reglas del dominio, el sistema debe cumplir un conjunto de restricciones de calidad expresadas en atributos de la norma ISO/IEC 25010 [5]. La aplicación debe responder al ochenta por ciento de las consultas de catálogo en menos de dos segundos, soportar cien usuarios concurrentes en horario pico, resistir intentos básicos de inyección SQL y de cross-site scripting según el Top 10 de OWASP [6], y ofrecer una interfaz accesible al nivel AA de las guías WCAG 2.1 [7].

Las restricciones no funcionales aparecen aquí con nombre propio porque más adelante se convertirán en criterios de aceptación de la rúbrica del PFC. Un aprendiz que las tenga presentes desde la Unidad I tomará decisiones —elegir imágenes con `alt` descriptivo, parametrizar las consultas SQL, escribir pruebas de carga— que un lector distraído postergará hasta que sea tarde.

1.3 Modelo conceptual de BiblioUTEQ

El modelo conceptual describe las entidades del dominio en un nivel de abstracción independiente de la tecnología con que se implementarán. La Figura 1.1 muestra el diagrama entidad-relación inicial, y las subsecciones que siguen describen cada entidad y sus responsabilidades. Esta separación inicial —modelo antes que tabla, entidad antes que anotación JPA— es una recomendación directa del enfoque orientado al dominio [4].

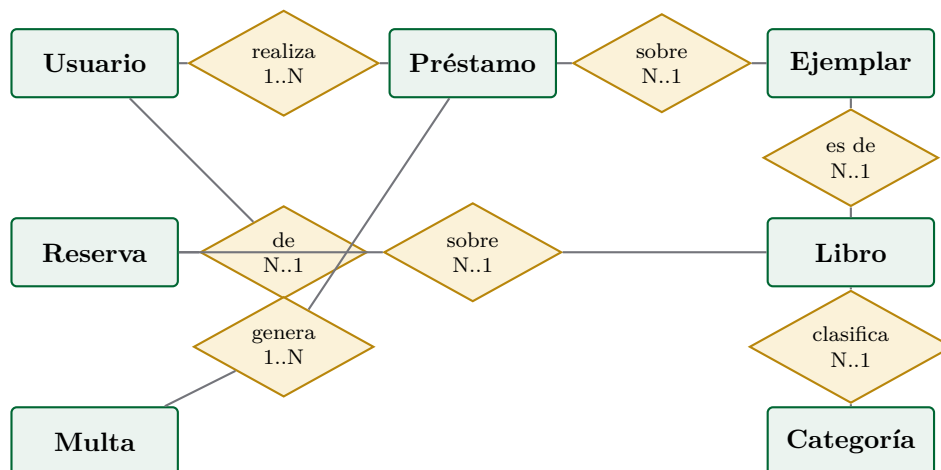


Figura 1.1: Diagrama entidad-relación inicial de BiblioUTEQ. Las cardinalidades finales se refinan al pasar del modelo conceptual al físico en la Unidad III.

1.3.1 Entidades y sus responsabilidades

El sistema se articula sobre siete entidades cuya responsabilidad se enuncia antes de definir sus atributos. Esta secuencia —responsabilidad primero, esquema después— refleja el consejo de diseño dirigido por el dominio de escribir el porqué antes que el qué [4].

La entidad **Libro** representa la obra intelectual: título, autor, editorial, ISBN, año. Una obra existe en el catálogo aun cuando no haya ejemplares físicos disponibles, y una operación de reserva se hace sobre el libro, no sobre el ejemplar.

La entidad **Ejemplar** representa la copia física. Un mismo libro tiene típicamente varios ejemplares con códigos de barras distintos y estados propios —disponible, prestado, reservado, en reparación, perdido—. El préstamo se materializa sobre un ejemplar concreto, no sobre el libro abstracto.

La entidad **Usuario** identifica a la persona que interactúa con el sistema, sea lector, bibliotecario, catalogador, director o administrador. Contiene los datos de contacto, el rol asignado y el estado

de bloqueo por multa. No incluye la contraseña en claro; el capítulo de seguridad de la Unidad II detalla cómo se almacena.

La entidad **Préstamo** registra el hecho histórico: un usuario retiró un ejemplar, con fecha de retiro, fecha de vencimiento calculada y fecha de devolución efectiva. Cuando el ejemplar regresa, el préstamo no se elimina; simplemente se cierra. La distinción entre *préstamo activo* y *préstamo cerrado* vive en el estado, no en la existencia de la fila.

La entidad **Reserva** registra la intención de un usuario de retirar un libro cuando ninguno de sus ejemplares esté disponible. La cola FIFO de reservas se resuelve automáticamente al momento de la devolución.

La entidad **Multa** registra el importe generado por un préstamo devuelto tarde. Se calcula con base en la regla R4 en el momento de cerrar el préstamo, y bloquea al usuario hasta ser pagada.

La entidad **Categoría** clasifica los libros por área temática y sirve como dimensión de los reportes de rotación por materia. Es una jerarquía simple en la primera versión y puede evolucionar a un árbol en versiones posteriores.

1.3.2 Casos de uso críticos

Sobre este modelo se enuncian cuatro casos de uso que dominan la superficie funcional del sistema y que atravesarán, uno u otro, cada capítulo del libro. La lista no es exhaustiva; es el mínimo con el que el aprendiz reconoce la utilidad del sistema.

El caso de uso *registrar préstamo* valida que el usuario esté sin bloqueo, verifica el límite de préstamos activos, comprueba que el ejemplar esté disponible, calcula la fecha de vencimiento según el rol del usuario, y persiste el préstamo dejando el ejemplar en estado **prestado**. Es el caso que más reglas de negocio concentra.

El caso de uso *devolver ejemplar* recibe un código de ejemplar, localiza el préstamo activo asociado, compara la fecha actual con la de vencimiento, genera la multa cuando corresponda según la regla R4, cierra el préstamo y libera el ejemplar. Si existe una reserva pendiente sobre el libro, notifica al primer usuario en la cola.

El caso de uso *reservar libro* verifica que no haya ejemplares disponibles, comprueba que el usuario no exceda su límite de reservas activas, y encola la reserva. Si más tarde se libera un ejemplar, la reserva se convierte en una notificación con un plazo de 48 horas para el retiro.

El caso de uso *reporte de rotación* agrega, por rango de fechas y por categoría, la cantidad de préstamos y la duración promedio del préstamo. Es un caso de solo lectura, computacionalmente más pesado que los anteriores, y se convertirá en el ejemplo canon del procedimiento almacenado en la Unidad III.

1.4 Arquitectura general y decisiones iniciales

Problema a resolver

Antes de escribir la primera línea de código el equipo debe fijar decisiones arquitectónicas que afectarán todas las decisiones posteriores. Elegir mal en este momento —o no elegir en absoluto— convierte el proyecto en una acumulación de parches. La pregunta operativa es: **qué decisiones dejan cerradas la Unidad 0, y cómo se documentan para que el equipo las respete.**

La respuesta se apoya en un instrumento sencillo y probado: los **registros de decisión de arquitectura** o ADR, propuestos originalmente por Nygard como bitácora ligera y trazable de

las opciones tomadas por el equipo [8]. Cada decisión relevante —estilo arquitectónico, base de datos, mecanismo de autenticación, formato de identificadores— vive en un archivo pequeño bajo `docs/adr/` con un formato uniforme que declara el contexto, la decisión, sus consecuencias y su estado.

El compendio adopta esta práctica de forma explícita desde la Unidad 0. El aprendiz que trabaje sobre BiblioUTEQ producirá al menos tres ADR en la primera entrega y uno adicional en cada entrega posterior. Los capítulos que introducen decisiones nuevas —por ejemplo, JPA versus procedimientos almacenados en la Unidad III— indicarán explícitamente qué ADR debe redactarse.

1.4.1 Estilo arquitectónico: cliente monopágina más API REST

BiblioUTEQ se construye como una aplicación de tipo **single-page application** respaldada por una API REST sin estado. El navegador carga una aplicación Angular que consume, mediante peticiones HTTP autenticadas, un conjunto de recursos publicados por un servidor Spring Boot. La separación clara entre cliente y servidor obedece a las cuatro restricciones clásicas del estilo REST propuestas por Fielding [9]: interfaz uniforme, ausencia de estado en el servidor, cacheable donde corresponda y sistema por capas.

Esa elección tiene consecuencias. Habilita —y hasta obliga a— desplegar frontend y backend de forma independiente, obliga a resolver desde el primer día la política CORS, y saca del servidor toda la lógica de presentación. Es una carga aparente más alta al principio, pero produce un sistema más escalable y más fácil de auditar en la etapa de servicios web [10].

1.4.2 Pilas del servidor aceptadas por la asignatura

Aunque el proyecto de referencia del libro utiliza Spring Boot como pila del servidor, la asignatura acepta tres pilas equivalentes en el PFC: Spring Boot con Java, ASP.NET Core con C# y PHP moderno con Postgres. La equivalencia no es solo técnica sino también evaluativa: la rúbrica del PFC califica atributos de calidad —seguridad, mantenibilidad, testabilidad— independientes del lenguaje.

Esta apertura persigue dos propósitos. Reconoce la diversidad real del mercado laboral ecuatoriano, donde los tres ecosistemas conviven, y evita el sesgo de una única pila que empobrece al egresado. En el compendio, los capítulos que introducen concepto —inyección de dependencias, transacciones, autenticación— llevan primero la explicación agnóstica de la pila y después la realización en cada una, en el orden Spring Boot, ASP.NET Core y PHP.

1.4.3 Base de datos: PostgreSQL como estándar

Para todo el compendio, la base de datos de referencia es PostgreSQL en su versión 16 o superior. La decisión descansa en tres criterios: es software libre con licencia abierta, ofrece cumplimiento estricto de SQL estándar y tiene soporte robusto para procedimientos almacenados en `plpgsql`, requisito de la política híbrida de acceso a datos que se introduce en la Unidad III. Otras opciones —SQL Server, MySQL o MariaDB— son admisibles en el PFC siempre que el equipo justifique la desviación mediante un ADR.

En BiblioUTEQ, la base recibirá el esquema `biblioteca` y se poblará con datos iniciales pseudo-realistas: cien libros, doscientos ejemplares, cincuenta usuarios de prueba con distintos roles. El script de población vivirá en el mismo repositorio, versionado, y será idempotente —el aprendiz puede ejecutarlo dos veces sin romper el estado.

1.5 Estructura del repositorio (monorepo)

BiblioUTEQ se organiza como un **monorepo**: un único repositorio Git contiene el código del frontend Angular, el código del backend Spring Boot, los scripts de base de datos, la documentación y la configuración de despliegue. La elección facilita al aprendiz recorrer el sistema completo sin saltar entre repositorios, y simplifica la trazabilidad entre commits del frontend y del backend en un mismo cambio funcional [11].

1.5.1 Convención de carpetas

La Figura 1.2 muestra la estructura de carpetas de nivel superior, con la función de cada una. Las profundidades mayores se detallan en cada capítulo cuando aparezca la tecnología correspondiente.

Estructura de directorios de BiblioUTEQ

```
1 bibliteq/
2 |-- backend-java/           # Spring Boot 3 (pila de referencia)
3 |   |-- src/main/java/
4 |   |-- src/main/resources/
5 |   |-- src/test/java/
6 |   |-- pom.xml
7 |   `-- Dockerfile
8 |-- backend-aspnet/         # ASP.NET Core 8 (pila alternativa)
9 |-- backend-php/            # PHP 8.3 (pila alternativa)
10 |-- frontend/              # Angular 18
11 |   |-- src/app/
12 |   |-- src/styles.css      # incluye estilos.css compartido
13 |   |-- package.json
14 |   `-- angular.json
15 |-- db/
16 |   |-- schema.sql          # DDL versionado
17 |   |-- datos-iniciales.sql # datos pseudo-realistas
18 |   `-- procs/              # procedimientos almacenados
19 |-- docs/
20 |   |-- adr/                # registros de decision de arquitectura
21 |   |-- arquitectura/        # diagramas C4 y de despliegue
22 |   |-- basedatos/          # MER, diccionario y CATALOGO-SP.md
23 |   |-- requerimientos/     # RF y RNF trazables
24 |   `-- entregas/           # PFC: 1A, 1B, 2A, 2B
25 |-- .github/                # workflows de CI
26 |-- docker-compose.yml      # base de datos + servicios locales
27 |-- README.md
28 `-- LICENSE
```

Figura 1.2: Estructura de carpetas de nivel superior del monorepo BiblioUTEQ.

El listado anterior fija cuatro decisiones que guían el resto del libro. Cada backend se aloja en su propia carpeta, lo que permite al equipo de PFC elegir su pila sin tocar las demás. El frontend es único y agnóstico del backend: consume la API REST por HTTP. Los artefactos de base de datos son un ciudadano de primera clase del repositorio —no viven en una carpeta oculta— porque el equipo los versionará con la misma disciplina que el código. La documentación convive con el código, no en un wiki paralelo, por congruencia con el enfoque *docs as code* recomendado por Bass y coautores [2].

1.5.2 Convención de nombres de paquetes

En el backend Java, la raíz de paquetes es `ec.uteq.biblioteq`. La organización sigue el patrón de *package by feature* —no *package by layer*— porque agrupa por característica de negocio y hace visible el dominio en el árbol de paquetes.

Bajo la raíz, cada agregado del dominio recibe un paquete: `libro`, `ejemplar`, `usuario`, `prestamo`, `reserva`, `multa` y `categoria`. Dentro de cada uno se ubican las clases de dominio, el repositorio, el servicio, el controlador REST y los objetos de transferencia. La sección de seguridad tiene su propio paquete `security`, y la configuración transversal vive en `config`. Esta convención hace que un aprendiz que abra el proyecto por primera vez encuentre en el árbol de carpetas el mismo mapa mental que aparece en el diagrama de dominio.

1.5.3 Convención de ramas y etiquetas Git

El equipo trabaja con dos ramas de larga vida —`main` y `develop`— y ramas de trabajo tipo `feature/<nombre-corto>` que nacen de `develop` y se integran mediante *pull request*. Cada entrega del PFC se marca con una etiqueta Git firmada que sigue el estándar de **versionado semántico** [12]: la Entrega 1A obtiene la etiqueta `v0.1.0`, la 1B `v0.2.0`, la 2A `v0.9.0-rc` y la 2B `v1.0.0`. Las correcciones posteriores a una entrega, si las hubiese, incrementan el número de *patch*.

La disciplina de etiquetado tiene una consecuencia práctica que el docente evaluará: cualquier estado del sistema en el momento de una entrega debe poder reconstruirse exactamente clonando el repositorio y ejecutando `git checkout v0.2.0`. Esta reproducibilidad es la base de la evaluación objetiva y coincide con las buenas prácticas de citación de software recomendadas por FORCE11 [13].

1.6 Herramientas del entorno de desarrollo

El compendio parte del supuesto de que el aprendiz dispone de un entorno de desarrollo unificado. La homogeneidad reduce la fricción inicial y libera al docente para atender problemas de fondo en lugar de diferencias de configuración. La Tabla 1.2 resume las herramientas requeridas con la versión de referencia verificada al inicio del período lectivo.

Tabla 1.2: *Herramientas del entorno de desarrollo con versiones de referencia.*

Herramienta	Versión	Rol en el proyecto
Java (Temurin JDK)	21 LTS o 25 LTS	Ejecución del backend Spring Boot
Node.js	20 LTS	Ejecución de Angular y <i>tooling</i> del frontend
Angular CLI	18.x	Andamiaje del proyecto frontend
PostgreSQL	16 o 18	Base de datos relacional
Docker Engine	24 o superior	Contenedores para servicios locales
Docker Compose	Plugin v2	Orquestación de la base y otros servicios
Git	2.40 o superior	Control de versiones distribuido
IntelliJ IDEA Ultimate	2024.x	IDE de referencia para Java y Angular
Visual Studio Code	1.90 o superior	Editor auxiliar y único para ASP.NET Core

1.6.1 IDE de referencia y editores auxiliares

El IDE de referencia para el compendio es IntelliJ IDEA Ultimate, cuya licencia académica gratuita cubre a los estudiantes de la carrera. IntelliJ ofrece soporte de primer nivel para el ecosistema Java, integra Angular con el mismo motor de análisis y expone acciones de refactor automático que respaldan las buenas prácticas de diseño que se enseñan en el libro [14].

Visual Studio Code se utiliza como editor auxiliar para tareas puntuales —edición de Markdown, YAML, scripts SQL de datos— y como IDE único para el backend ASP.NET Core cuando el equipo elige esa pila. La combinación de ambos cubre la variabilidad de estándares laborales que el egresado encontrará al terminar la carrera.

1.6.2 Servicios locales

La base de datos y otros servicios auxiliares se ejecutan localmente mediante Docker Compose. Esta elección evita conflictos de versión entre desarrolladores y garantiza que el arranque de un entorno nuevo se reduzca a clonar el repositorio y ejecutar `docker compose up`. El archivo `docker-compose.yml` en la raíz del monorepo declara los servicios esperados y sus parámetros de red.

En etapas posteriores del libro, cuando se introduzcan caché distribuida y colas de mensajes, esos servicios se sumarán al mismo archivo. Esta acumulación progresiva —en lugar de un *stack* inicial imponente— respeta el principio de introducir cada componente cuando exista una necesidad concreta que lo justifique.

1.6.3 Comandos de arranque

Con las herramientas instaladas, el aprendiz clona el repositorio, entra al directorio del backend Java y ejecuta `mvn spring-boot:run`; en otra terminal, entra al directorio del frontend Angular y ejecuta `ng serve`. La aplicación completa queda accesible en `http://localhost:4200` con la API REST respondiendo en `http://localhost:8080`.

Este flujo, tan corto, es el resultado deliberado de las convenciones fijadas en las secciones anteriores. Un aprendiz sin guía terminaría inventando decisiones ad hoc en cada arranque; con las convenciones documentadas, el proyecto se comporta como un producto de ingeniería.

1.7 Cómo se organiza cada capítulo del libro

Cada capítulo del compendio sigue una estructura uniforme cuya razón de ser es didáctica y no cosmética. Un aprendiz que reconozca los bloques del capítulo puede navegar con seguridad, saltar cuando le convenga y volver cuando lo necesite. La uniformidad reduce la carga cognitiva del acto de leer y libera esfuerzo para el acto de aprender.

1.7.1 El método del libro

El método se resume en cuatro pasos que se repiten a lo largo del libro. Primero, el capítulo abre planteando explícitamente el problema que se resolverá en el capítulo, con una descripción breve y una caja de *Problema* destacada visualmente.

Segundo, se explica la técnica que resuelve el problema, con las referencias bibliográficas que respaldan cada afirmación no trivial. La explicación privilegia la comprensión por sobre la exhaustividad: se cubre lo esencial y se remite al aprendiz a la referencia primaria cuando desee ampliar.

Tercero, se muestra el listado de código que materializa la técnica. Cada listado va precedido por un párrafo que anuncia qué se pretende resolver y qué funciones o clases aparecerán; y se

cierra con un párrafo posterior que explica cómo el código resuelve el problema. Los listados que muestran archivos completos llevan como comentario inicial la ruta corta del archivo dentro del proyecto —por ejemplo, `// org.uteq.biblioteq.services.LibroService.java`—, lo que permite al aprendiz ubicar en un vistazo dónde debe colocar el archivo.

Cuarto, el capítulo cierra conectando la pieza recién construida con el estado global del proyecto BiblioUTEQ. Este cierre es lo que distingue una guía técnica de un libro sobre un sistema: mantiene la coherencia narrativa del compendio y evita la fragmentación en tutoriales inconexos [15].

1.7.2 Cajas temáticas y su significado

El libro utiliza siete tipos de cajas visualmente distintas con un significado fijo que conviene fijar aquí. La caja de *Problema* enmarca el planteamiento que abre cada bloque temático. La caja de *Definición* formaliza un término que se usará más adelante; el aprendiz que dude puede volver a ella. La caja de *Ejemplo* muestra un caso pequeño completo, casi siempre resuelto en el flujo del texto.

La caja de *Código* envuelve un listado con su ruta y su lenguaje. La caja de *Ejercicio* propone una tarea al aprendiz con criterios de evaluación. Las cajas de *Consejo*, *Advertencia* y *Peligro* marcan buenas prácticas, errores frecuentes y decisiones irreversibles respectivamente, con colores que van del verde al rojo.

Un lector atento notará que las cajas no son un adorno tipográfico: sirven a la función de navegación. Un capítulo que carezca de caja de *Problema* al abrir, o que no cierre con la conexión al proyecto, contradice el método del libro y debe ser reportado al autor como un defecto editorial.

1.8 Cierre del encuadre

Con este capítulo cero, el proyecto BiblioUTEQ deja de ser un enunciado genérico y se convierte en un dominio con siete entidades, siete reglas de negocio, cuatro casos de uso principales, un modelo relacional inicial, un monorepo con carpetas definidas, un flujo de Git con convenciones de ramas y etiquetas, una pila de referencia con versiones verificadas y un método de trabajo declarado. Todos esos elementos permanecen constantes a lo largo de las cuatro unidades; lo que cambia es qué piezas se agregan al mismo repositorio y qué problema se resuelve en cada capítulo.

La Unidad I comienza en el navegador. El aprendiz maquetará la página de aterrizaje del catálogo de BiblioUTEQ, escribirá la hoja de estilos base que compartirán el resto de las páginas del sistema, y programará la interacción inicial con JavaScript moderno. La entrega 1A del PFC, al cierre de la Unidad I, requerirá exactamente esos artefactos en el repositorio, etiquetados con `v0.1.0`. El aprendiz que llegue con este encuadre leído tendrá el mapa del territorio que está a punto de recorrer.

Parte I

Fundamentos del cliente web

Unidad I — En preparación

Los capítulos de esta unidad —HTML5 semántico, CSS3 con diseño responsivo, JavaScript moderno y programación asíncrona— y la Guía del PFC Entrega 1A se entregan en el bloque siguiente del desarrollo del libro.

Parte II

Unidad 2 — En preparación

Unidad 2 — En preparación

El contenido de esta unidad se entrega en el bloque siguiente del desarrollo del libro.

Parte III

Unidad 3 — En preparación

Unidad 3 — En preparación

El contenido de esta unidad se entrega en el bloque siguiente del desarrollo del libro.

Parte IV

Unidad 4 — En preparación

Unidad 4 — En preparación

El contenido de esta unidad se entrega en el bloque siguiente del desarrollo del libro.

Bibliografía

- [1] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Upper Saddle River, NJ, USA: Prentice Hall, 2017. ISBN: 978-0-13-449416-6.
- [2] Len Bass, Paul Clements y Rick Kazman. *Software Architecture in Practice*. 4th. Boston, MA, USA: Addison-Wesley Professional, 2021. ISBN: 978-0-13-688609-9.
- [3] Ian Sommerville. *Software Engineering*. 10th. Boston, MA, USA: Pearson Education, 2015. ISBN: 978-0-13-394303-0.
- [4] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley Professional, 2003. ISBN: 978-0-321-12521-7.
- [5] ISO/IEC. *ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization. 2011. URL: <https://www.iso.org/standard/35733.html>.
- [6] OWASP Foundation. *OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks*. 2021. URL: <https://owasp.org/Top10/>.
- [7] W3C Web Accessibility Initiative. *Web Content Accessibility Guidelines (WCAG) 2.1*. World Wide Web Consortium (W3C) Recommendation. Jun. de 2018. URL: <https://www.w3.org/TR/WCAG21/>.
- [8] Michael Nygard. *Documenting Architecture Decisions*. Cognitect Blog. Nov. de 2011. URL: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>.
- [9] Roy Thomas Fielding. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doct. University of California, Irvine, 2000. URL: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [10] Mark Richards y Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol, CA, USA: O'Reilly Media, 2020. ISBN: 978-1-4920-4345-4.
- [11] Scott Chacon y Ben Straub. *Pro Git*. 2nd. Berkeley, CA, USA: Apress, 2014. ISBN: 978-1-4842-0076-6. URL: <https://git-scm.com/book/en/v2>.
- [12] Tom Preston-Werner. *Semantic Versioning 2.0.0*. 2013. URL: <https://semver.org/spec/v2.0.0.html>.
- [13] Arfon M. Smith, Daniel S. Katz, Kyle E. Niemeyer y FORCE11 Software Citation Working Group. «Software citation principles». En: *PeerJ Computer Science* 2 (2016), e86. DOI: 10.7717/peerj-cs.86.
- [14] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. 2nd. Boston, MA, USA: Addison-Wesley Professional, 2018. ISBN: 978-0-13-475759-9.
- [15] Hironori Washizaki, ed. *SWEBOK Guide v4.0: Software Engineering Body of Knowledge*. Piscataway, NJ, USA: IEEE Computer Society, 2024. ISBN: 978-0-7381-9741-9. URL: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>.